

Hive Framework -- Training Tutorial

Table of Contents

1. [What is Hive?](#1-what-is-hive)
2. [Why Use Hive?](#2-why-use-hive)
3. [Core Concepts](#3-core-concepts)
4. [Major Functions & API](#4-major-functions--api)
5. [Cell Lifecycle](#5-cell-lifecycle)
6. [Multi-Cell Example: Weather Alert System](#6-multi-cell-example-weather-alert-system)
7. [Advanced Patterns](#7-advanced-patterns)
8. [Common Pitfalls](#8-common-pitfalls)
9. [Comparison with Other Frameworks](#9-comparison-with-other-frameworks)

1. What is Hive?

Hive is a dependency injection (DI) and lifecycle management framework for Go applications, developed by the Cilium project. It provides:

- Automatic constructor wiring (no manual New() call chains)
- Ordered startup and shutdown of components
- Configuration management via struct tags
- Health checking and status reporting
- Modular composition of application features

Repository: github.com/cilium/hive

Used by: Cilium Agent, Cilium Operator, Hubble, and other Cilium ecosystem projects.

2. Why Use Hive?

Problem: Manual Wiring at Scale

// Without Hive -- 50+ components manually wired:

```
func main() {
    cfg := loadConfig()
    log := newLogger(cfg)
    db := newDatabase(cfg, log)
    cache := newCache(cfg, log, db)
    auth := newAuth(cfg, log, db)
    api := newAPI(cfg, log, db, cache, auth)
    metrics := newMetrics(cfg, log)

    // Must start in correct order:
    db.Start()
    cache.Start()
    auth.Start()
    api.Start()
    metrics.Start()

    // Must stop in reverse order:
    metrics.Stop()
    api.Stop()
    auth.Stop()
    cache.Stop()
    db.Stop()
}
```

Problems:

1. Manual ordering is error-prone
2. Adding a new dependency requires editing main()
3. Testing requires recreating the entire chain
4. No clear boundary between components

Solution: Hive

```
func main() {
    h := hive.New(
        DatabaseCell,
        CacheCell,
        AuthCell,
        APICell,
        MetricsCell,
    )
    h.Run() // Hive figures out order, starts, waits, stops
}
```

3. Core Concepts

3.1 Cell

A Cell is the fundamental unit of modularity. It declares what it provides and what it needs.

```
var MyCell = cell.Module(  
    "my-component",    // unique identifier  
    "Human-readable desc", // description  
  
    cell.Provide(NewMyService), // register constructors  
    cell.Config(DefaultConfig), // register configuration  
    cell.Invoke(startBackground), // side-effects at startup  
)
```

3.2 Provide

cell.Provide registers a constructor. Hive calls it once, injects its parameters, and makes the return value available to other cells.

```
cell.Provide(NewDatabase) // func NewDatabase(cfg Config, log *slog.Logger) *Database  
  
// Hive will:  
// 1. Find a Config and *slog.Logger in the DI container  
// 2. Call NewDatabase(cfg, log)  
// 3. Store the returned *Database for other cells to use
```

3.3 Invoke

cell.Invoke registers a function for side-effects. It runs AFTER all Provide constructors.

```
cell.Invoke(startMetricsServer)  
// func startMetricsServer(lc cell.Lifecycle, metrics *Metrics) { ... }
```

3.4 Config

cell.Config registers a configuration struct. Fields are populated from flags, env vars, or defaults.

```
type Config struct {  
    Port    int    `mapstructure:"port" default:"8080"`  
    Timeout string `mapstructure:"timeout" default:"30s"`  
}  
  
cell.Config(Config{}) // registers --port and --timeout flags
```

3.5 Lifecycle

cell.Lifecycle lets you register start/stop hooks that run in dependency order.

```
func NewServer(lc cell.Lifecycle, cfg Config) *Server {
    s := &Server{port: cfg.Port}
    lc.Append(cell.Hook{
        OnStart: func(ctx cell.HookContext) error {
            return s.Listen()
        },
        OnStop: func(ctx cell.HookContext) error {
            return s.Close()
        },
    })
    return s
}

---
```

4. Major Functions & API

4.1 Top-Level Functions

Function	Purpose
<code>`hive.New(cells...)`</code>	Create a new Hive with the given cells
<code>`h.Run()`</code>	Start all cells, wait for signal, stop all
<code>`h.Start(log, ctx)`</code>	Start all cells (manual control)
<code>`h.Stop(log, ctx)`</code>	Stop all cells in reverse order
<code>`h.RegisterFlags(flags)`</code>	Register all cell configs as CLI flags
<code>`h.Populate(log)`</code>	Resolve DI graph without starting

4.2 Cell Module Functions

Function	Purpose
<code>`cell.Module(id, desc, cells...)`</code>	Group cells into a named module
<code>`cell.Group(cells...)`</code>	Group cells without naming (anonymous)
<code>`cell.Provide(ctors...)`</code>	Register constructors (public output)
<code>`cell.ProvidePrivate(ctors...)`</code>	Register constructors (module-scoped output)
<code>`cell.Invoke(fns...)`</code>	Register side-effect functions
<code>`cell.Config(defaultCfg)`</code>	Register configuration struct
<code>`cell.Decorate(fn)`</code>	Modify a dependency for this module's scope

4.3 Lifecycle API

Function	Purpose
<code>lc.Append(hook)</code>	Register start/stop hook
<code>cell.Hook{OnStart, OnStop}</code>	Hook struct with start/stop callbacks
<code>cell.HookContext</code>	Context with timeout for start/stop

4.4 Health API

Function	Purpose
<code>health.OK(msg)</code>	Report healthy status
<code>health.Degraded(msg, err)</code>	Report degraded status
<code>health.Stopped(msg)</code>	Report stopped status

4.5 Job API

Function	Purpose
<code>job.Group</code>	Container for background jobs
<code>g.Add(job.OneShot(...))</code>	Run-once background task
<code>g.Add(job.Timer(...))</code>	Periodic background task
<code>g.Add(job.Observer(...))</code>	React to stream events

4.6 Special DI Tags

```
type MyParams struct {
    cell.In // marks this as DI parameter struct

    Required *Database // must exist, or Hive fails to start
    Optional *Cache   `optional:"true"` // nil if not provided
    Named    *Logger   `name:"audit"` // disambiguate by name
}
```

5. Cell Lifecycle

Startup Sequence

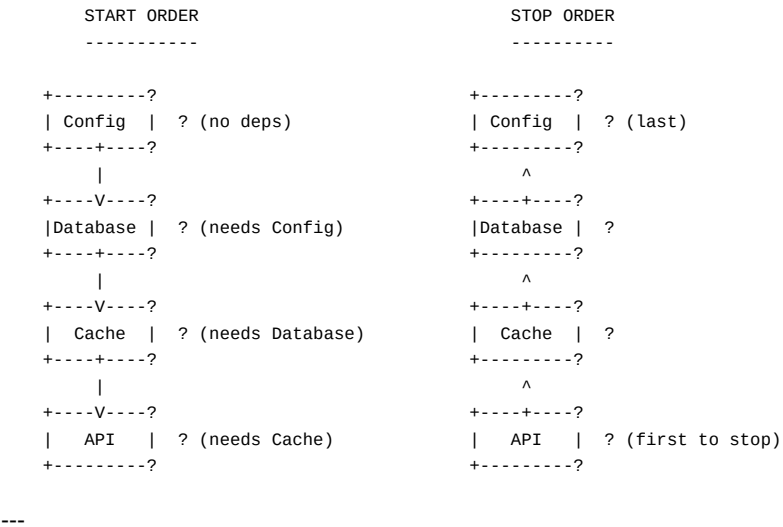
- hive.New(cells...)
+-- Registers all cells in the DI container

```
2. h.Start()
+-- Resolve DI graph (topological sort by dependencies)
+-- Instantiate all Provide constructors (in dependency order)
+-- Run all Invoke functions
+-- Execute OnStart hooks (in dependency order):
    Cell A (no deps) -> Cell B (depends on A) -> Cell C (depends on B)

3. Application runs...

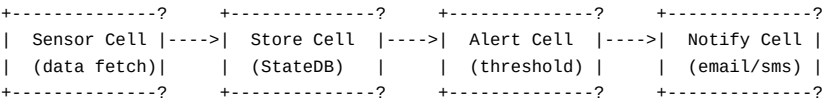
4. h.Stop()
+-- Execute OnStop hooks (in REVERSE order):
    Cell C -> Cell B -> Cell A
```

Visualization



6. Multi-Cell Example: Weather Alert System

Let's build a weather monitoring system with 4 cells:



Full Implementation

```
package main

import (
    "context"
    "fmt"
    "log/slog"
    "math/rand"
    "sync"
    "time"

    "github.com/cilium/hive"
    "github.com/cilium/hive/cell"
    "github.com/cilium/hive/job"
)

// =====
// CELL 1: Sensor Cell -- Fetches weather data periodically
// =====

var SensorCell = cell.Module(
    "sensor",
    "Weather sensor data collector",

    cell.Config(SensorConfig{}),
    cell.Provide(NewSensor),
)

type SensorConfig struct {
    PollInterval time.Duration `mapstructure:"sensor-poll-interval" default:"5s"`
    Location      string      `mapstructure:"sensor-location" default:"Seattle"`
}

// WeatherReading is the data produced by the sensor.
type WeatherReading struct {
    Temperature float64
    Humidity     float64
    WindSpeed    float64
    Location     string
    Timestamp    time.Time
}

// Sensor fetches weather data on a timer.
type Sensor struct {
    cfg      SensorConfig
    log      *slog.Logger
    store    *Store // injected dependency from Store Cell
    cancelFn context.CancelFunc
}

type sensorParams struct {
```

```

cell.In

Log      *slog.Logger
Lifecycle cell.Lifecycle
Config   SensorConfig
Store    *Store      // ? DI: comes from Store Cell
}

func NewSensor(p sensorParams) *Sensor {
    s := &Sensor{
        cfg:   p.Config,
        log:    p.Log,
        store: p.Store,
    }

    p.Lifecycle.Append(cell.Hook{
        OnStart: func(ctx cell.HookContext) error {
            innerCtx, cancel := context.WithCancel(context.Background())
            s.cancelFn = cancel
            go s.pollLoop(innerCtx)
            s.log.Info("Sensor started", "location", s.cfg.Location,
                "interval", s.cfg.PollInterval)
            return nil
        },
        OnStop: func(ctx cell.HookContext) error {
            s.cancelFn()
            s.log.Info("Sensor stopped")
            return nil
        },
    })

    return s
}

func (s *Sensor) pollLoop(ctx context.Context) {
    ticker := time.NewTicker(s.cfg.PollInterval)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return
        case <-ticker.C:
            reading := WeatherReading{
                Temperature: 15 + rand.Float64()*25, // 15-40°C
                Humidity:      30 + rand.Float64()*60, // 30-90%
                WindSpeed:    rand.Float64() * 100,   // 0-100 km/h
                Location:    s.cfg.Location,
                Timestamp:   time.Now(),
            }
            s.store.RecordReading(reading)
            s.log.Debug("Sensor reading", "temp", reading.Temperature)
        }
    }
}

```



```

    }
}

// =====
// CELL 2: Store Cell -- In-memory state store (like StateDB)
// =====

var StoreCell = cell.Module(
    "store",
    "Weather data store",

    cell.Config(StoreConfig{}),
    cell.Provide(NewStore),
)

type StoreConfig struct {
    MaxReadings int `mapstructure:"store-max-readings" default:"100"`
}

// Store holds recent weather readings and notifies subscribers.
type Store struct {
    mu          sync.RWMutex
    readings    []WeatherReading
    maxSize     int
    listeners   []chan WeatherReading
    log         *slog.Logger
}

func NewStore(log *slog.Logger, cfg StoreConfig) *Store {
    return &Store{
        readings: make([]WeatherReading, 0, cfg.MaxReadings),
        maxSize:  cfg.MaxReadings,
        log:       log,
    }
}

// RecordReading stores a reading and notifies all subscribers.
func (s *Store) RecordReading(r WeatherReading) {
    s.mu.Lock()
    if len(s.readings) >= s.maxSize {
        s.readings = s.readings[1:] // drop oldest
    }
    s.readings = append(s.readings, r)
    listeners := make([]chan WeatherReading, len(s.listeners))
    copy(listeners, s.listeners)
    s.mu.Unlock()

    // Notify all watchers (non-blocking)
    for _, ch := range listeners {
        select {
        case ch <- r:
        }
    }
}

```

```

        default:
    }
}

// Watch returns a channel that receives new readings.
func (s *Store) Watch() <-chan WeatherReading {
    ch := make(chan WeatherReading, 10)
    s.mu.Lock()
    s.listeners = append(s.listeners, ch)
    s.mu.Unlock()
    return ch
}

// GetLatest returns the most recent reading.
func (s *Store) GetLatest() (WeatherReading, bool) {
    s.mu.RLock()
    defer s.mu.RUnlock()
    if len(s.readings) == 0 {
        return WeatherReading{}, false
    }
    return s.readings[len(s.readings)-1], true
}

// =====
// CELL 3: Alert Cell -- Monitors readings and triggers alerts
// =====

var AlertCell = cell.Module(
    "alert",
    "Weather alert evaluator",

    cell.Config(AlertConfig{}),
    cell.Provide(NewAlertEvaluator),
)

type AlertConfig struct {
    TempThreshold float64 `mapstructure:"alert-temp-threshold" default:"35.0"`
    WindThreshold float64 `mapstructure:"alert-wind-threshold" default:"80.0"`
}

// Alert represents a triggered weather alert.
type Alert struct {
    Severity string // "warning", "critical"
    Message  string
    Reading  WeatherReading
}

// AlertEvaluator watches the store and produces alerts.
type AlertEvaluator struct {
    cfg    AlertConfig
    log    *slog.Logger

```

```

    store    *Store
    notifier *Notifier
    cancelFn context.CancelFunc
}

type alertParams struct {
    cell.In

    Log      *slog.Logger
    Lifecycle cell.Lifecycle
    Config    AlertConfig
    Store     *Store    // ? DI: from Store Cell
    Notifier  *Notifier // ? DI: from Notify Cell
}

func NewAlertEvaluator(p alertParams) *AlertEvaluator {
    a := &AlertEvaluator{
        cfg:      p.Config,
        log:      p.Log,
        store:    p.Store,
        notifier: p.Notifier,
    }

    p.Lifecycle.Append(cell.Hook{
        OnStart: func(ctx cell.HookContext) error {
            innerCtx, cancel := context.WithCancel(context.Background())
            a.cancelFn = cancel
            go a.watchLoop(innerCtx)
            a.log.Info("Alert evaluator started",
                "tempThreshold", a.cfg.TempThreshold,
                "windThreshold", a.cfg.WindThreshold)
            return nil
        },
        OnStop: func(ctx cell.HookContext) error {
            a.cancelFn()
            a.log.Info("Alert evaluator stopped")
            return nil
        },
    })

    return a
}

func (a *AlertEvaluator) watchLoop(ctx context.Context) {
    ch := a.store.Watch()

    for {
        select {
        case <-ctx.Done():
            return
        case reading := <-ch:
            a.evaluate(reading)
        }
    }
}

```

```

    }
}

func (a *AlertEvaluator) evaluate(r WeatherReading) {
    if r.Temperature > a.cfg.TempThreshold {
        alert := Alert{
            Severity: "critical",
            Message:  fmt.Sprintf("HEAT ALERT: %.1f°C exceeds %.1f°C threshold",
                                r.Temperature, a.cfg.TempThreshold),
            Reading: r,
        }
        a.log.Warn("Alert triggered", "severity", alert.Severity, "msg", alert.Message)
        a.notifier.Send(alert)
    }

    if r.WindSpeed > a.cfg.WindThreshold {
        alert := Alert{
            Severity: "warning",
            Message:  fmt.Sprintf("WIND ALERT: %.1f km/h exceeds %.1f km/h threshold",
                                r.WindSpeed, a.cfg.WindThreshold),
            Reading: r,
        }
        a.log.Warn("Alert triggered", "severity", alert.Severity, "msg", alert.Message)
        a.notifier.Send(alert)
    }
}

// =====
// CELL 4: Notify Cell -- Sends alerts via email/SMS/webhook
// =====

var NotifyCell = cell.Module(
    "notify",
    "Alert notification dispatcher",

    cell.Config(NotifyConfig{}),
    cell.Provide(NewNotifier),
)

type NotifyConfig struct {
    Channel  string `mapstructure:"notify-channel" default:"console"`
    Recipient string `mapstructure:"notify-recipient" default:"admin@example.com"`
}

// Notifier dispatches alerts to configured channels.
type Notifier struct {
    cfg NotifyConfig
    log *slog.Logger
    mu  sync.Mutex

    // Track sent alerts for deduplication

```

```

    lastAlert time.Time
}

func NewNotifier(log *slog.Logger, cfg NotifyConfig) *Notifier {
    log.Info("Notifier initialized", "channel", cfg.Channel, "recipient", cfg.Recipient)
    return &Notifier{
        cfg: cfg,
        log: log,
    }
}

func (n *Notifier) Send(alert Alert) {
    n.mu.Lock()
    defer n.mu.Unlock()

    // Simple dedup: don't send more than 1 alert per 10 seconds
    if time.Since(n.lastAlert) < 10*time.Second {
        return
    }
    n.lastAlert = time.Now()

    switch n.cfg.Channel {
    case "console":
        fmt.Printf("\n? [%s] %s\n    Location: %s | Time: %s\n\n",
            alert.Severity, alert.Message,
            alert.Reading.Location,
            alert.Reading.Timestamp.Format("15:04:05"))
    case "email":
        n.log.Info("Sending email alert",
            "to", n.cfg.Recipient,
            "severity", alert.Severity)
        // email.Send(n.cfg.Recipient, alert.Message)
    }
}

// =====
// MAIN -- Compose all cells into a Hive
// =====

func main() {
    h := hive.New(
        // Order doesn't matter! Hive resolves dependencies automatically.
        NotifyCell,    // provides *Notifier
        StoreCell,     // provides *Store
        AlertCell,     // needs *Store + *Notifier
        SensorCell,    // needs *Store
    )

    // Run blocks until SIGINT/SIGTERM, then stops all cells gracefully.
    h.Run()
}

```

Dependency Resolution

Hive resolves this automatically:

```
NotifyCell (no deps)      -> instantiates *Notifier FIRST
|
StoreCell (no deps)       -> instantiates *Store SECOND
|
+-----?
|               |
AlertCell (needs *Store + *Notifier) | -> instantiates THIRD
|               |
SensorCell (needs *Store) -----? -> instantiates FOURTH
```

```
START ORDER: Notifier -> Store -> AlertEvaluator -> Sensor
STOP ORDER:  Sensor -> AlertEvaluator -> Store -> Notifier
```

Running the Example

```
# With defaults:
go run main.go
```

```
# With custom config:
go run main.go \
  --sensor-poll-interval=2s \
  --sensor-location=Mumbai \
  --alert-temp-threshold=38.0 \
  --notify-channel=console
```

Expected Output

```
INFO Notifier initialized channel=console recipient=admin@example.com
INFO Sensor started location=Seattle interval=5s
INFO Alert evaluator started tempThreshold=35.0 windThreshold=80.0
```

```
? [critical] HEAT ALERT: 38.2°C exceeds 35.0°C threshold
  Location: Seattle | Time: 14:23:15
```

```
? [warning] WIND ALERT: 85.3 km/h exceeds 80.0 km/h threshold
  Location: Seattle | Time: 14:23:20
```

```
^C
INFO Sensor stopped
INFO Alert evaluator stopped
```

```
---
```

7. Advanced Patterns

7.1 Private vs Public Provide

```
var MyCell = cell.Module("my-cell", "...",
    // Public: available to ANY cell in the hive
    cell.Provide(NewPublicService),

    // Private: only available to cells WITHIN this module
    cell.ProvidePrivate(newInternalHelper),
)
```

Use case: Expose Table[*Frontend] (read-only) publicly, but keep RWTable[*Frontend] private to prevent external writes.

7.2 Optional Dependencies

```
type myParams struct {
    cell.In

    Cache *Cache `optional:"true"` // nil if no cell provides *Cache
}

func NewService(p myParams) *Service {
    if p.Cache != nil {
        // use cache
    } else {
        // fallback to direct queries
    }
}
```

7.3 Named Dependencies (Multiple of Same Type)

```
// Provider:
cell.Provide(cell.Named("primary", NewPrimaryDB))
cell.Provide(cell.Named("replica", NewReplicaDB))

// Consumer:
type params struct {
    cell.In
    Primary *Database `name:"primary"`
    Replica *Database `name:"replica"`
}
```

7.4 Lifecycle Hooks with Interfaces

```
// If your struct implements cell.HookInterface, no need for explicit Append:
type Server struct { ... }

func (s *Server) Start(cell.HookContext) error { return s.listen() }
func (s *Server) Stop(cell.HookContext) error { return s.close() }

func NewServer(lc cell.Lifecycle, ...) *Server {
    s := &Server{...}
    lc.Append(s) // calls s.Start() on start, s.Stop() on stop
    return s
}
```

7.5 Job Groups (Background Workers)

```
cell.Invoke(func(g job.Group, store *Store) {
    // OneShot: runs once, completes
    g.Add(job.OneShot("init-data", func(ctx context.Context, health cell.Health) error {
        health.OK("Loading initial data")
        return store.LoadFromDisk()
    })))

    // Timer: runs periodically
    g.Add(job.Timer("cleanup", func(ctx context.Context) error {
        return store.PruneOld(24 * time.Hour)
    }, 1*time.Hour))
})
```

7.6 Promises (Async Initialization)

```
// Provider: resolve later when ready
cell.Provide(func() (promise.Promise[*Database], promise.Resolver[*Database]) {
    return promise.New[*Database]()
})

// Async resolver:
cell.Invoke(func(resolve promise.Resolver[*Database], cfg Config) {
    go func() {
        db, err := connectWithRetry(cfg)
        if err != nil {
            resolve.Reject(err)
        } else {
            resolve.Resolve(db)
        }
    }()
})

// Consumer: blocks until resolved
cell.Invoke(func(dbPromise promise.Promise[*Database]) {
    go func() {
```



```

        db, err := dbPromise.Await(ctx)
        // use db
    }()
})

```

7.7 Module Composition (Nested Cells)

```

var ParentCell = cell.Module("parent", "Parent module",
    ChildACell,
    ChildBCell,
    cell.Provide(NewParentService),
)

var ChildACell = cell.Module("child-a", "...",
    cell.Provide(NewChildA),
)

// ChildA's outputs are visible to Parent and siblings

---
```

8. Common Pitfalls

[X] Import Cycles

```

// pkg/a imports pkg/b
// pkg/b imports pkg/a  -> COMPILE ERROR

// Fix: Use interfaces at the composition level
cell.Invoke(func(a *A, b *B) {
    a.SetB(b) // wire at runtime, no import needed
})

```

[X] Forgetting cell.In

```

// WRONG: Hive doesn't know this is a DI params struct
type myParams struct {
    Log *slog.Logger
}

// RIGHT:
type myParams struct {
    cell.In           // ? MUST include this
    Log *slog.Logger
}

```

[X] Starting work in Constructor

```
// WRONG: don't start goroutines in constructor
func NewService(log *slog.Logger) *Service {
    s := &Service{}
    go s.run() // ? BAD: runs before other cells are ready
    return s
}

// RIGHT: use lifecycle hook
func NewService(lc cell.Lifecycle, log *slog.Logger) *Service {
    s := &Service{}
    lc.Append(cell.Hook{
        OnStart: func(ctx cell.HookContext) error {
            go s.run() // ? GOOD: runs after all deps are started
            return nil
        },
    })
    return s
}
```

[X] Circular Dependencies

```
// A needs B, B needs A -> Hive panics at startup
// Fix: Break cycle with interface, Invoke wiring, or Promise
```

[X] Blocking in OnStart

```
// WRONG: blocks other cells from starting
OnStart: func(ctx cell.HookContext) error {
    return s.connectWithRetry() // may take 30s
}

// RIGHT: start async, use health reporting
OnStart: func(ctx cell.HookContext) error {
    go func() {
        if err := s.connectWithRetry(); err != nil {
            health.Degraded("connection failed", err)
        } else {
            health.OK("connected")
        }
    }()
    return nil
}
```

9. Comparison with Other Frameworks

Feature	Hive	Wire (Google)	fx (Uber)	Manual DI
-----	-----	-----	-----	-----
DI resolution	Runtime	Compile-time codegen	Runtime	Manual
Lifecycle mgmt	Built-in (ordered)	None	Built-in	Manual
Configuration	Built-in (flags/env)	None	None	Manual
Health checks	Built-in	None	None	Manual
Job scheduling	Built-in (job.Group)	None	None	Manual
Error on missing dep	Startup (fast-fail)	Compile-time	Startup	Runtime crash
Module scoping	Private/Public provide	Package-level	Module-level	Package-level
Go-specific	Yes (generics, tags)	Yes (codegen)	Reflect-heavy	Pure Go
Learning curve	Medium	Low	Medium	None

When to use Hive:

- Applications with 10+ interdependent components
- Need ordered startup/shutdown
- Want built-in configuration management
- Need health reporting and background job scheduling
- Cilium ecosystem projects

When NOT to use Hive:

- Small CLIs or scripts (< 5 components)
- Libraries (DI is for applications)
- When compile-time safety is paramount (use Wire instead)

Appendix: Quick Reference Card

+-----?		
	HIVE CHEAT SHEET	
+-----+		
CREATE HIVE:	h := hive.New(CellA, CellB, CellC)	
RUN (blocking):	h.Run()	
START/STOP:	h.Start(log, ctx) / h.Stop(log, ctx)	
DEFINE CELL:	var Cell = cell.Module("id", "desc",	
	cell.Provide(Constructor),	
	cell.Config(DefaultConfig{}),	
	cell.Invoke(SideEffect),	

```
|
|
| )
|
| CONSTRUCTOR:    func New(p Params) *MyType { ... }
|
| PARAMS STRUCT:  type Params struct {
|                  cell.In
|                  Dep1 *TypeA
|                  Dep2 *TypeB `optional:"true"`
|                  }
|
| LIFECYCLE:       lc.Append(cell.Hook{
|                  OnStart: func(cell.HookContext)error
|                  OnStop:  func(cell.HookContext)error
|                  })
|
| CONFIG:          type Config struct {
|                  Field Type `mapstructure:"flag"`
|                  }
|
| PRIVATE PROVIDE: cell.ProvidePrivate(fn)
| (module-scoped)
|
| JOBS:            g.Add(job.OneShot("name", fn))
|                  g.Add(job.Timer("name", fn, interval))
|
| HEALTH:          health.OK("msg")
|                  health.Degraded("msg", err)
|
|
+-----?

```